

Chapter 5: Large and Fast

Exploiting Memory Hierarchy

Complete Notes with Exercises

Contents

I	Memory Hierarchy Fundamentals	2
1	Introduction (5.1): The Dream of Unlimited Fast Memory	2
1.1	The Principle of Locality	2
1.2	Memory Hierarchy Structure	2
1.3	Key Terminology	2
2	Memory Technologies (5.2)	3
2.1	SRAM – Static Random Access Memory	3
2.2	DRAM – Dynamic Random Access Memory	3
2.3	Flash Memory	3
2.4	Magnetic Disk	3
3	The Basics of Caches (5.3)	4
3.1	Direct-Mapped Cache	4
3.2	Address Breakdown	4
3.3	Write Policies	4
4	Measuring and Improving Cache Performance (5.4)	5
4.1	Performance Equations	5
4.2	Reducing Miss Rate: Associativity	5
4.3	Reducing Miss Penalty: Multilevel Caches	5
4.4	LRU Replacement	5
5	A Common Framework for Memory Hierarchy (5.8)	6
5.1	The Four Questions	6
5.2	The Three Cs Model	6
II	Exercises and Solutions	7
6	Exercise 5.1: Matrix Locality	7
7	Exercise 5.2: Cache Behavior	8
8	Exercise 5.3: Cache Size Calculations	9
9	Exercise 5.5: Cache Analysis	10
10	Exercise 5.9: Optimal Block Size	11
11	Exercise 5.10: Cache Performance	12
12	Exercise 5.12: Multilevel Caches	13

Part I

Memory Hierarchy Fundamentals

1 Introduction (5.1): The Dream of Unlimited Fast Memory

“Ideally one would desire an indefinitely large memory capacity such that any particular...word would be immediately available... We are...forced to recognize the possibility of constructing a hierarchy of memories.”

— A. W. Burks, H. H. Goldstine, and J. von Neumann (1946)

Memory hierarchies create the **illusion** of having unlimited amounts of fast memory by exploiting the **principle of locality**.

1.1 The Principle of Locality

Temporal Locality (Locality in Time)

If a data item is referenced, it will tend to be referenced **again soon**.

Example: A textbook you’re currently reading (loop variables, frequently called functions).

Spatial Locality (Locality in Space)

If a data item is referenced, items whose **addresses are close by** will tend to be referenced soon.

Example: Sequential array access, instruction fetching.

1.2 Memory Hierarchy Structure

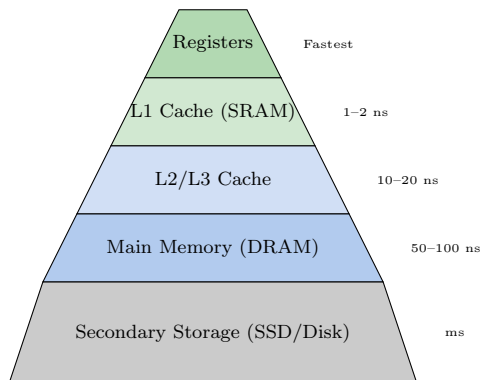


Figure 1: Memory hierarchy pyramid.

1.3 Key Terminology

Hit and Miss

- **Hit:** Data found in upper level of hierarchy.
- **Miss:** Data NOT found; must access lower level.
- **Hit Rate:** Fraction of accesses found in upper level.
- **Miss Rate:** $1 - \text{Hit Rate}$
- **Hit Time:** Time to access upper level.
- **Miss Penalty:** Time to fetch block from lower level.

2 Memory Technologies (5.2)

2.1 SRAM – Static Random Access Memory

- Used for: **Caches**
- Access time: $\sim 0.5\text{--}2.5$ ns
- No refresh needed (6 transistors per bit)
- Most expensive per bit

2.2 DRAM – Dynamic Random Access Memory

- Used for: **Main memory**
- Access time: $\sim 50\text{--}70$ ns
- Requires periodic **refresh** (1 transistor + 1 capacitor)
- $\sim 100\times$ slower than SRAM, but $\sim 50\times$ cheaper
- Modern: DDR4/DDR5 with burst transfers

2.3 Flash Memory

- Used for: **SSDs, mobile devices**
- Access time: $\sim 5\text{--}50$ μs
- Nonvolatile, rugged (no moving parts)
- Limited write cycles (**wear leveling** required)

2.4 Magnetic Disk

- Used for: **Large storage**
- Access time: $\sim 5\text{--}20$ ms
- Components: Platters, tracks, sectors, read/write head
- Three stages: Seek $\rightarrow$ Rotational Latency $\rightarrow$ Transfer

3 The Basics of Caches (5.3)

3.1 Direct-Mapped Cache

Direct-Mapped Cache

Each memory block maps to **exactly one** cache location:

$$\text{Cache Block} = (\text{Block Address}) \bmod (\text{Number of Blocks})$$

3.2 Address Breakdown

For a 64-bit address with 2^n blocks and 2^m words per block:

Tag	Index	Block Offset	Byte Offset
$64 - n - m - 2$ bits	n bits	m bits	2 bits

Hit Conditions: Tag matches AND Valid bit = 1.

3.3 Write Policies

- **Write-Through:** Write to both cache and memory.
- **Write-Back:** Write only to cache; write to memory on replacement.
- **Write Buffer:** Queue for pending writes (avoids stalls).

4 Measuring and Improving Cache Performance (5.4)

4.1 Performance Equations

CPU Time

$$\text{CPU Time} = (\text{CPU cycles} + \text{Memory-stall cycles}) \times \text{Cycle time}$$

Average Memory Access Time (AMAT)

$$\text{AMAT} = \text{Hit time} + (\text{Miss rate} \times \text{Miss penalty})$$

4.2 Reducing Miss Rate: Associativity

- **Direct-Mapped:** 1 location per block (fast, high miss rate).
- **n-way Set Associative:** n locations per set.
- **Fully Associative:** Any location (low miss rate, slow).

Key insight: 1-way $\rightarrow$ 2-way gives **20–30% miss rate reduction**.

4.3 Reducing Miss Penalty: Multilevel Caches

- **L1 Cache:** Focus on minimizing **hit time**.
- **L2/L3 Cache:** Focus on minimizing **miss rate**.

4.4 LRU Replacement

Replace the block that has been **unused for the longest time**. Complex to implement for high associativity (>4 -way).

5 A Common Framework for Memory Hierarchy (5.8)

5.1 The Four Questions

Common Framework

1. **Where can a block be placed?** Direct-mapped, set-associative, fully associative.
2. **How is a block found?** Indexing, limited search, full search, page table.
3. **What block is replaced?** LRU or Random.
4. **How are writes handled?** Write-through or write-back.

5.2 The Three Cs Model

Three Cs

- **Compulsory Misses:** First access to a block. *Reduce by:* Increase block size.
- **Capacity Misses:** Cache too small. *Reduce by:* Increase cache size.
- **Conflict Misses:** Multiple blocks compete for same set. *Reduce by:* Increase associativity.

Note: Fully associative caches have **no conflict misses**.

Part II

Exercises and Solutions

6 Exercise 5.1: Matrix Locality

Given code (C, row-major storage):

```
for (I=0; I<8; I++)
  for (J=0; J<8000; J++)
    A[I][J] = B[I][0] + A[J][I];
```

5.1.1: 64-bit integers in a 16-byte cache block

$$\frac{16 \text{ bytes}}{8 \text{ bytes/integer}} = \boxed{2 \text{ integers}}$$

5.1.2: Temporal locality (C)

- **B[I][0]**: Referenced 8000 times for each value of I. ✓
- **I, J**: Loop variables referenced repeatedly. ✓

5.1.3: Spatial locality (C)

- **A[I][J]**: Accessed sequentially in row-major order (J varies fastest). ✓

(A[J][I] has poor spatial locality – different rows each iteration.)

Matlab code (column-major storage):

5.1.4: Temporal locality (Matlab)

- **B(I,0)**: Same as C – referenced 8000 times per I. ✓
- **I, J**: Loop variables. ✓

5.1.5: Spatial locality (Matlab)

- **A(J,I)**: In Matlab (column-major), varying J accesses consecutive memory. ✓

(A(I,J) now has poor spatial locality.)

5.1.6: Cache blocks needed

Matrix A is 8000×8000, B is 8000×1 (only column 0 used).

Elements referenced:

- **A[I][J]**: 8 rows × 8000 cols = 64,000 elements (write side)
- **A[J][I]**: 8000 rows × 8 cols = 64,000 elements (read side)
- **B[I][0]**: 8 elements

Unique elements: $\approx 64,000 + 64,000 + 8 = 128,008$ (some overlap)

Blocks (16 bytes = 2 integers):

- **Matlab:** $\approx 64,004$ blocks (good column locality)
- **C:** $\approx 64,004$ blocks (good row locality for A[I][J])

7 Exercise 5.2: Cache Behavior

Address sequence: 0x03, 0xb4, 0x2b, 0x02, 0xbf, 0x58, 0xbe, 0x0e, 0xb5, 0x2c, 0xba, 0xfd

5.2.1: Direct-mapped, 16 one-word blocks

Index = Address mod 16 (4 bits), Tag = Address / 16

Addr	Binary	Tag	Index	Hit/Miss
0x03	0000 0011	0	3	Miss
0xb4	1011 0100	11	4	Miss
0x2b	0010 1011	2	11	Miss
0x02	0000 0010	0	2	Miss
0xbf	1011 1111	11	15	Miss
0x58	0101 1000	5	8	Miss
0xbe	1011 1110	11	14	Miss
0x0e	0000 1110	0	14	Miss
0xb5	1011 0101	11	5	Miss
0x2c	0010 1100	2	12	Miss
0xba	1011 1010	11	10	Miss
0xfd	1111 1101	15	13	Miss

Result: 0 hits, 12 misses (all compulsory misses).

5.2.2: Direct-mapped, 8 blocks, 2-word blocks

Index = (Address / 2) mod 8 (3 bits), Offset = Address mod 2, Tag = Address / 16

Addr	Tag	Index	Offset	Hit/Miss
0x03	0	1	1	Miss
0xb4	11	2	0	Miss
0x2b	2	5	1	Miss
0x02	0	1	0	Hit (block 0x02-0x03 loaded)
0xbf	11	7	1	Miss
0x58	5	4	0	Miss
0xbe	11	7	0	Hit
0x0e	0	7	0	Miss (replaces 0xbe)
0xb5	11	2	1	Hit
0x2c	2	6	0	Miss
0xba	11	5	0	Miss (replaces 0x2b)
0xfd	15	6	1	Miss (replaces 0x2c)

Result: 3 hits, 9 misses.

8 Exercise 5.3: Cache Size Calculations

5.3.1: 32 KiB cache, 2-word blocks

- Data: $32 \text{ KiB} = 32,768 \text{ bytes} = 4096 \text{ words}$ (64-bit words = 8 bytes)
- Block size: 2 words = 16 bytes
- Number of blocks: $32768/16 = 2048 \text{ blocks}$
- Block offset: $\log_2(2) = 1 \text{ bit}$
- Byte offset: $\log_2(8) = 3 \text{ bits}$
- Index: $\log_2(2048) = 11 \text{ bits}$
- Tag: $64 - 11 - 1 - 3 = 49 \text{ bits}$
- Per block: Valid (1) + Tag (49) + Data ($2 \times 64 = 128$) = 178 bits
- Total: $2048 \times 178 = \boxed{364,544 \text{ bits}}$

5.3.2: 64 KiB cache, 16-word blocks

- Data: $64 \text{ KiB} = 65,536 \text{ bytes} = 8192 \text{ words}$
- Block size: 16 words = 128 bytes
- Number of blocks: $65536/128 = 512 \text{ blocks}$
- Block offset: $\log_2(16) = 4 \text{ bits}$
- Byte offset: $\log_2(8) = 3 \text{ bits}$
- Index: $\log_2(512) = 9 \text{ bits}$
- Tag: $64 - 9 - 4 - 3 = 48 \text{ bits}$
- Per block: Valid (1) + Tag (48) + Data ($16 \times 64 = 1024$) = 1073 bits
- Total: $512 \times 1073 = \boxed{549,376 \text{ bits}}$

Ratio: $549376/364544 = 1.51 \times$ (only 51% larger, not 100%, despite $2 \times$ data).

5.3.3: Why larger cache might be slower

- Larger block size $\rightarrow$ **higher miss penalty** (more data to transfer)
- Fewer blocks (512 vs 2048) $\rightarrow$ more **conflict misses**
- Miss penalty dominates if spatial locality is poor

9 Exercise 5.5: Cache Analysis

Given: Tag [63:10], Index [9:5], Offset [4:0]

5.5.1: Cache block size

Offset = 5 bits $\Rightarrow$ Block size = $2^5 = 32$ bytes = **4 words** (64-bit words).

5.5.2: Number of blocks

Index = 5 bits $\Rightarrow$ Number of blocks = $2^5 = \boxed{32 \text{ blocks}}$.

5.5.3: Bits ratio

- Data per block: $32 \times 8 = 256$ bits
- Tag: 54 bits, Valid: 1 bit
- Total per block: $256 + 54 + 1 = 311$ bits
- Total cache: $32 \times 311 = 9952$ bits
- Data only: $32 \times 256 = 8192$ bits
- Ratio: $9952/8192 = \boxed{1.215}$

10 Exercise 5.9: Optimal Block Size

Given: Base CPI = 1, 1.35 refs/instruction

5.9.1: Miss latency = $20 \times B$ cycles

- Miss rate decreases with block size, miss penalty = $20B$
- Total miss cycles = Miss rate $\times$ Miss penalty $\times 1.35$
- From table: Optimal is where (Miss rate $\times B$) is minimized
- **Answer: 32 bytes** (typical optimal for linear penalty)

5.9.2: Miss latency = $24 + B$ cycles

- Fixed component dominates for small B
- Larger blocks reduce miss rate without huge penalty increase
- **Answer: 64 bytes** (larger blocks preferred)

5.9.3: Constant miss latency

If miss latency is constant, minimize **miss rate** only.

- Larger blocks $\rightarrow$ lower miss rate (spatial locality)
- But too large $\rightarrow$ capacity issues
- **Answer: Largest block size before capacity issues**

11 Exercise 5.10: Cache Performance

Given: Main memory = 70 ns, 36% data memory instructions.

5.10.1: Clock rates from hit times

- P1: Hit time = 0.66 ns $\Rightarrow$ Clock rate = $1/0.66 = \boxed{1.52 \text{ GHz}}$
- P2: Hit time = 0.90 ns $\Rightarrow$ Clock rate = $1/0.90 = \boxed{1.11 \text{ GHz}}$

5.10.2: AMAT (in cycles)

AMAT = Hit time + Miss rate $\times$ Miss penalty

P1: Miss penalty = $70/0.66 = 106$ cycles

- Instruction: $1 + 0.08 \times 106 = 9.48$ cycles
- Data: $1 + 0.08 \times 106 = 9.48$ cycles
- Weighted: $1 + 0.08 \times 106 = \boxed{9.48 \text{ cycles}}$

P2: Miss penalty = $70/0.90 = 78$ cycles

- AMAT = $1 + 0.06 \times 78 = \boxed{5.68 \text{ cycles}}$

5.10.3: Total CPI

CPI = Base CPI + Memory stall cycles per instruction

P1:

- Instruction stalls: $1.0 \times 0.08 \times 106 = 8.48$
- Data stalls: $0.36 \times 0.08 \times 106 = 3.05$
- Total CPI = $1 + 8.48 + 3.05 = \boxed{12.53}$

P2:

- Instruction stalls: $1.0 \times 0.06 \times 78 = 4.68$
- Data stalls: $0.36 \times 0.06 \times 78 = 1.68$
- Total CPI = $1 + 4.68 + 1.68 = \boxed{7.36}$

P2 is faster despite lower clock rate due to lower miss rate.

12 Exercise 5.12: Multilevel Caches

Given: L1 miss rate = 7%, various L2 configurations.

5.12.1: CPI with different cache configurations

L1 only:

- $\text{CPI} = \text{Base} + \text{L1 misses} \times \text{Memory penalty}$
- $\text{CPI} = 1.5 + 0.07 \times 100 = \boxed{8.5}$

L1 + L2 direct-mapped:

- L2 miss rate = 35% (local)
- $\text{CPI} = 1.5 + 0.07 \times (15 + 0.35 \times 100)$
- $\text{CPI} = 1.5 + 0.07 \times (15 + 35) = 1.5 + 3.5 = \boxed{5.0}$

L1 + L2 8-way:

- L2 miss rate = 25% (local)
- $\text{CPI} = 1.5 + 0.07 \times (15 + 0.25 \times 100)$
- $\text{CPI} = 1.5 + 0.07 \times 40 = 1.5 + 2.8 = \boxed{4.3}$

With doubled memory time (200 cycles):

- L1 only: $1.5 + 0.07 \times 200 = 15.5$ (82% increase)
- L2 direct: $1.5 + 0.07 \times (15 + 0.35 \times 200) = 6.5$ (30% increase)
- L2 8-way: $1.5 + 0.07 \times (15 + 0.25 \times 200) = 5.05$ (17% increase)

L2 cache effectively **hides slow memory** effects.

5.12.2: Third-level cache

L3 with 50 cycles access, 13% miss rate:

- $\text{CPI} = 1.5 + 0.07 \times (15 + 0.35 \times (50 + 0.13 \times 100))$
- $\text{CPI} = 1.5 + 0.07 \times (15 + 0.35 \times 63) = 1.5 + 0.07 \times 37 = 4.09$

Yes, L3 provides better performance (4.09 vs 5.0).

Advantages of L3: Reduces main memory accesses, hides memory latency.

Disadvantages: Added complexity, power, cost, area.